Cross-Site Request Forgery (CSRF) in Web Security

1. Introduction

CSRF (Cross-Site Request Forgery) is a web security attack where a malicious website tricks a logged-in user into performing an unwanted action on another website without their knowledge.

Since browsers automatically include the user's session cookies with requests, the target website believes the request was made by the legitimate user.

---

2. Simple Example

Imagine you are logged into your bank website.

Step 1

You log in to **bank.com**.

Step 2

The bank stores your authenticated session in a browser cookie.

Step 3

Without logging out, you visit a malicious website.

Step 4

The malicious website secretly submits the following request:

```http
POST /transfer
Host: bank.com

amount=10000&toAccount=123456
```

Step 5

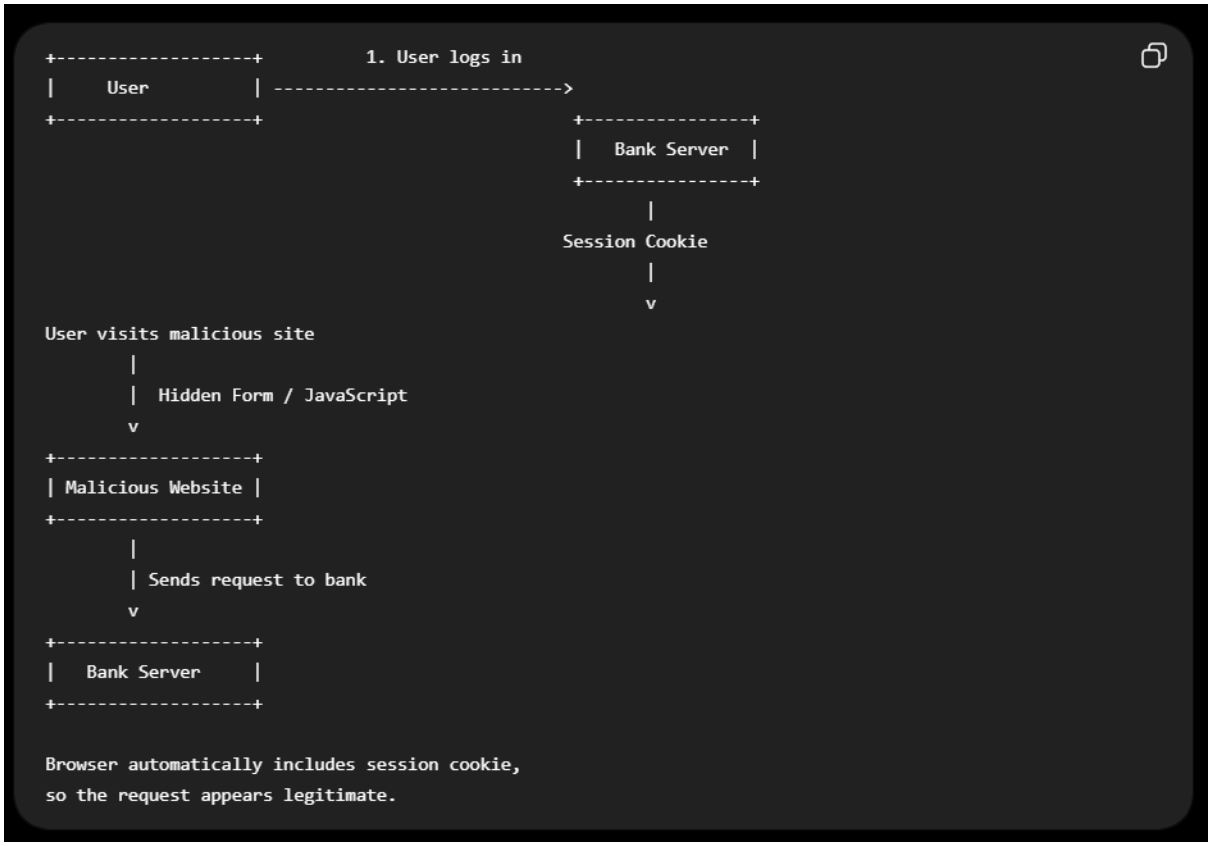
Your browser automatically sends the bank's session cookie along with the request.

The bank assumes that **you** initiated the transaction because the request contains a valid authenticated session.

Result

Money could be transferred from your account without your knowledge or permission.

3. How CSRF Works



4. Why Does It Happen?

CSRF occurs because browsers automatically send:

- Session Cookies
- Authentication Cookies

with every request to the same website.

The attacker **cannot directly read your sensitive information**, but they **can force your browser to send requests** on your behalf.

---

5. Example of a CSRF Attack

Without CSRF protection, an attacker can create a hidden HTML form.

```HTML
<form action="https://bank.com/transfer" method="POST">
    <input type="hidden" name="amount" value="5000">
    <input type="hidden" name="to" value="attacker">
</form>

<script>
document.forms[0].submit();
</script>
```

If the user is already logged into **bank.com**, the browser automatically sends the session cookie.

The bank processes the request as though it came from the legitimate user.

6. How to Prevent CSRF

6.1 CSRF Token (Most Common Method)

The server generates a unique random token for each user session.

Example:

```HTML
<input type="hidden"
       name="_csrf"
       value="a8f6d923bcde">
```

When the form is submitted:

```http
POST /transfer

amount=1000
to=12345
_csrf=a8f6d923bcde
```

The server validates the token.

If the token is missing or incorrect:

Why It Works

An attacker cannot obtain the correct CSRF token from the authenticated user's session, so forged requests fail validation.

6.2 SameSite Cookies

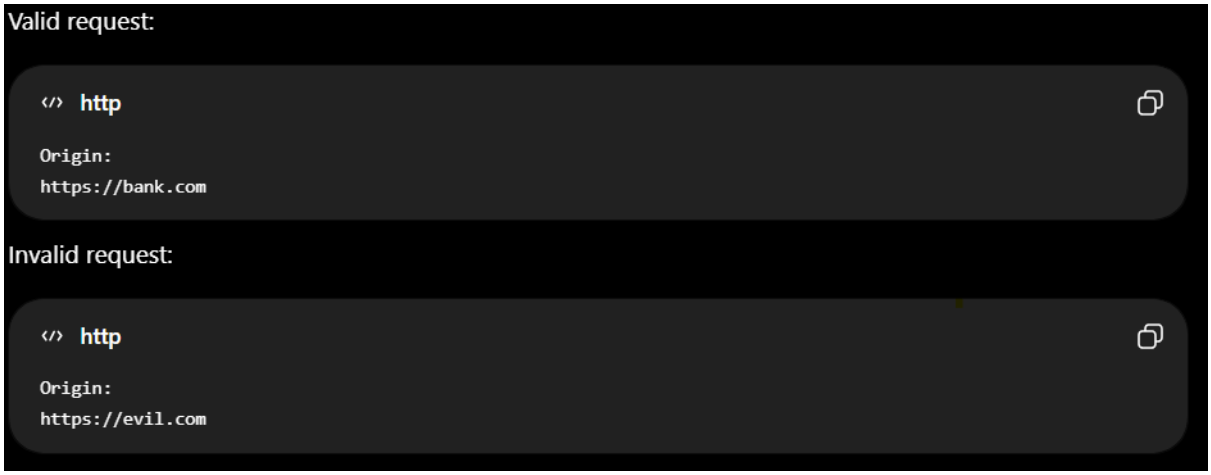
The server can configure cookies as follows:

This prevents browsers from sending cookies with many cross-site requests, significantly reducing the risk of CSRF attacks.

6.3 Verify Origin or Referer Headers

The server checks the request origin.

Valid request:



If the origin does not match the expected domain, the server rejects the request.

6.4 Re-authentication for Sensitive Operations

For highly sensitive actions, require the user to authenticate again.

Examples include:

- Changing Password
- Updating Email Address
- Bank Transfers
- Account Deletion

This adds an additional layer of security even if a forged request is attempted.

---

7. CSRF Protection in Spring Security

Spring Security enables CSRF protection by default for web applications that use **session-based authentication**.

Example configuration:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  SecurityFilterChain security(HttpSecurity http) throws Exception {
```

```
    http
        .csrf(Customizer.withDefaults());

    return http.build();
  }
}
```

Spring Security automatically:

- Generates a CSRF token
- Includes the token in forms
- Validates the token for state-changing HTTP requests

These requests include:

- POST
- PUT
- DELETE
- PATCH

---

8. Disabling CSRF Protection

CSRF protection can be disabled as follows:

```
http
  .csrf(csrf -> csrf.disable());
```

When Should CSRF Be Disabled?

Only when developing stateless REST APIs that use authentication mechanisms such as:

- JWT (JSON Web Token)
- OAuth Access Tokens

Since authentication tokens are manually supplied in request headers rather than automatically sent by the browser, CSRF attacks are generally not applicable in these scenarios.

9. CSRF vs XSS

Feature	CSRF	XSS
Full Form	Cross-Site Request Forgery	Cross-Site Scripting
Objective	Trick a user into performing an unwanted action	Inject malicious JavaScript into a trusted website
Requires Logged-in User	Yes	Not necessarily
Can Directly Read User Data	No	Yes
Uses Browser Session	Yes	May abuse the session after executing malicious code
Primary Protection	CSRF Tokens, SameSite Cookies, Origin Validation	Input Validation, Output Encoding, Content Security Policy (CSP)

10. Interview Answer (1–2 Minutes)

Question: What is CSRF?

Answer:

Cross-Site Request Forgery (CSRF) is a web security vulnerability in which a malicious website tricks an authenticated user's browser into sending unauthorized requests to another website where the user is already logged in. Because browsers automatically

include session cookies with requests, the server mistakenly believes the request is legitimate. To prevent CSRF attacks, applications typically use CSRF tokens, SameSite cookies, and validation of the Origin or Referer headers. In Spring Security, CSRF protection is enabled by default for session-based applications. It should generally remain enabled unless the application is a stateless REST API that uses JWT or another token-based authentication mechanism.

---

11. Key Takeaways

- CSRF stands for **Cross-Site Request Forgery**.
- It exploits the browser's automatic inclusion of session cookies.
- The attacker cannot directly read the user's data but can cause unauthorized actions.
- CSRF mainly affects **session-based authentication**.
- Effective protections include:
    - CSRF Tokens
    - SameSite Cookies
    - Origin/Referer Header Validation
    - Re-authentication for sensitive operations
- Spring Security enables CSRF protection by default.
- Stateless REST APIs using JWT typically disable CSRF because authentication tokens are not automatically sent by the browser.